

Document Object Model (DOM)

Unit - I

Dr. Narayan Joshi

Contents

- What is DOM?
- DOM view – Tree structure
- DOM Traversal
- DOM Manipulation
- Dynamic creation & removal of elements
- Manipulate contents and attributes
- Change styles and classes dynamically
- DOM event handling
- Event propagation and delegation

Learning Outcomes

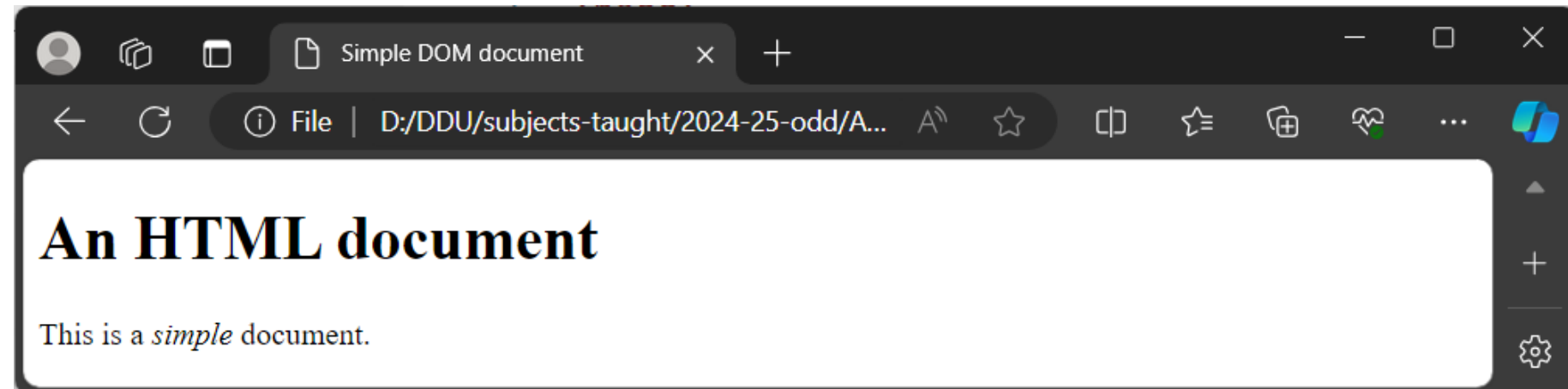
- This unit will
 - introduce the DOM,
 - look at how the DOM represents an HTML document in memory, and
 - how to use APIs to create dynamic web content and applications.

What is DOM?

- This is what the browser reads:

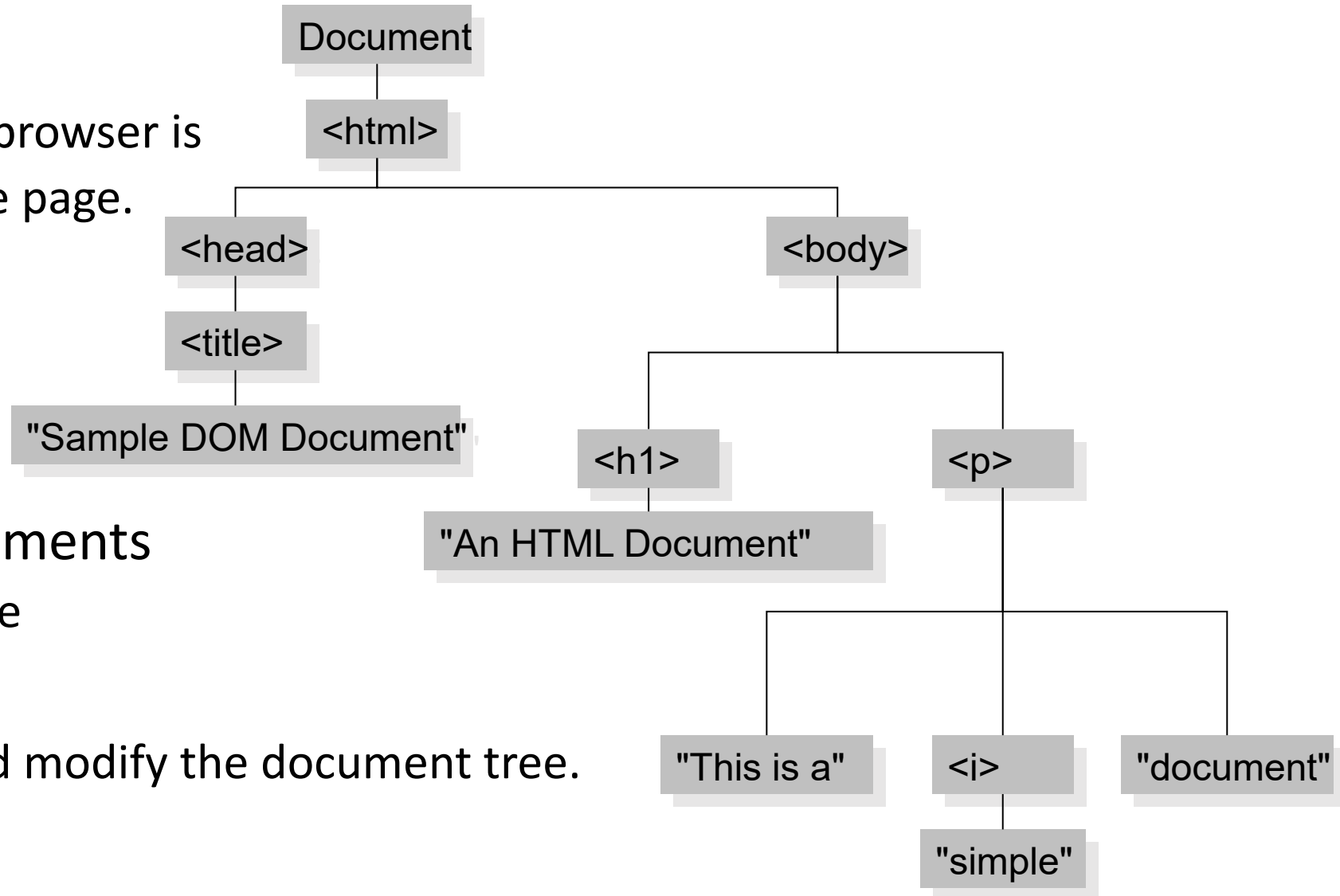
```
0-simple-page.html X
0-DOM > <> 0-simple-page.html > html
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Simple DOM document</title>
5  </head>
6  <body>
7  |   <h1>An HTML document</h1>
8  |   <p>This is a <i>simple</i> document.</p>
9  </body>
10 </html>
```

- This is what the browser displays on screen:



What is DOM?

- This is a drawing of
 - the model that the browser is
 - working with for the page.
- HTML document elements
 - form a tree structure
- DOM allows
 - scripts to access and modify the document tree.



What is DOM?

- What is model?
 - Collection of things
 - Connections (relationships) among those things
 - E.g. Entity-Relationship model represents two aspects:
 - Entities
 - Relationships (connections or network) among those entities
- Similarly, your web page (HTML document) consists of
 - Various HTML objects (such as: button, text, head, body, etc...)
- So, the web-browser builds a model of
 - various constituent objects of a web page (HTML Document).
 - Each HTML element becomes an object.
 - Each object has: attributes (properties), and methods

What is DOM?

- The Document Object Model (DOM) is
 - a programming interface for web documents.
 - It represents the page so that
 - programs can change the document structure, style, and content.
 - The DOM represents the document as nodes and objects; that way,
 - programming languages can interact with the page.
- A web page is a document that can be
 - either displayed in the browser window or as the HTML source.
 - In both cases, it is the same document but
 - the Document Object Model (DOM) representation allows it to be manipulated.
 - As an object-oriented representation of the web page,
 - it can be modified with a scripting language such as JavaScript.

What is DOM?

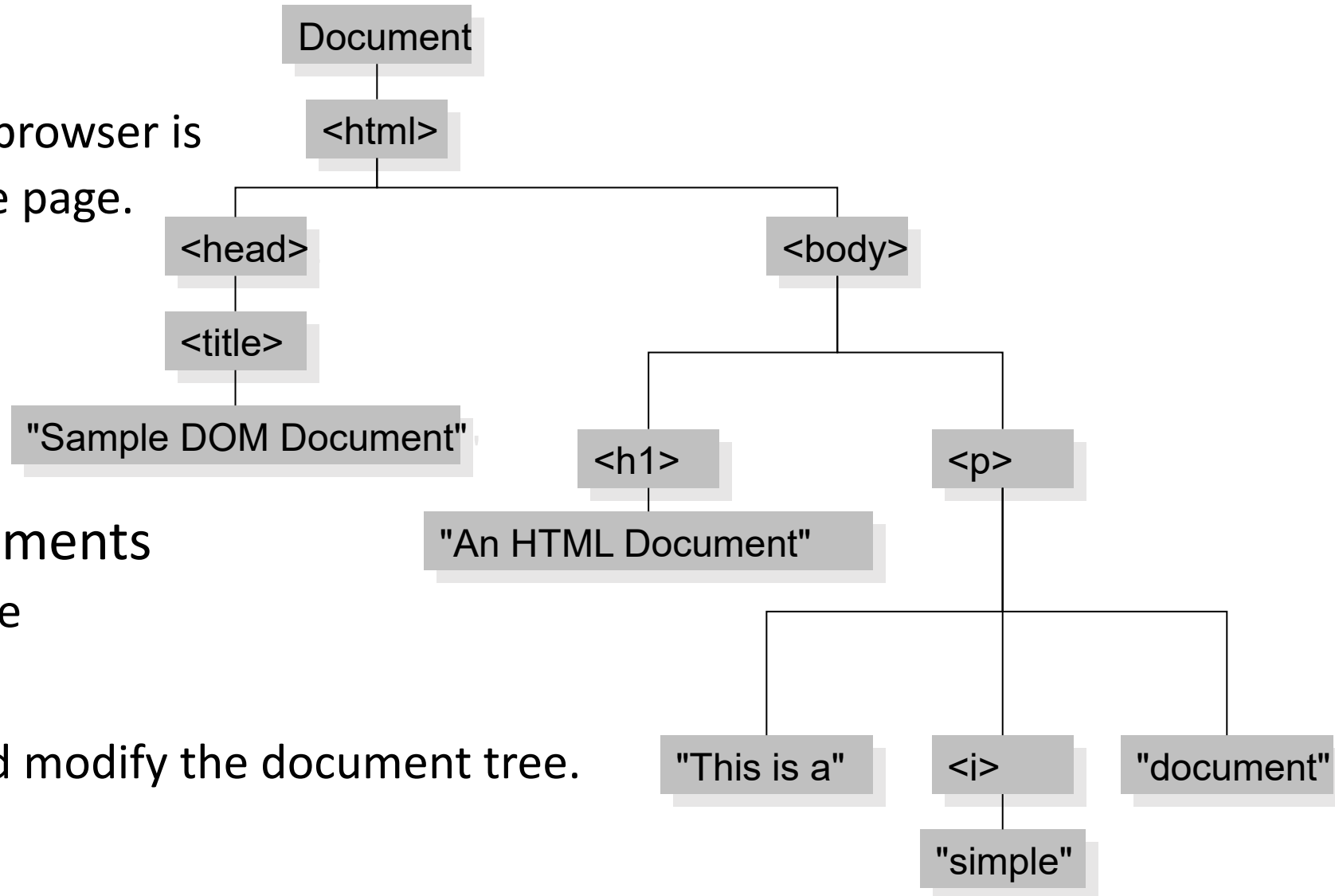
- The Document Object Model (DOM) is
 - an API that allows programs to interact with HTML (or XML) documents
- In typical browsers,
 - the JavaScript version of the API is provided via the document host object
 - W3C recommendations define standard DOM
- Several other browser host objects are informal, de facto standards
 - alert, prompt are examples

What is DOM?

- All of the properties, methods, and events
 - available for manipulating and creating web pages are organized into objects.
 - For example,
 - the document object that represents the document itself,
 - any table objects that implement the `HTMLTableElement` DOM interface for accessing HTML tables,
 - and so forth, are all objects.

What is DOM?

- This is a drawing of
 - the model that the browser is
 - working with for the page.
- HTML document elements
 - form a tree structure
- DOM allows
 - scripts to access and modify the document tree.



Why DOM?

- We can access the model, too!
 - the model is made available to scripts running in the browser, (not just the browser itself)
 - A script can find things out about the state of the page
 - A script can change things in response to events, including user requests
- Object's property's values can be changed.
 - It causes web page to change in-place.

DOM & Javascript

- The DOM is not a programming language, but without it,
 - the JavaScript language wouldn't have any model or notion of web pages, HTML documents, SVG documents, and their component parts.
 - The document as a whole (the head, tables within the document, table headers, text within the table cells, and all other elements in a document)
 - are parts of the document object model for that document.
 - They can all be accessed and manipulated using the DOM and a scripting language like JavaScript.
- The DOM is not part of the JavaScript language,
 - but is instead a Web API used to build websites.
 - JavaScript can also be used in other contexts.
 - For example, Node.js runs JavaScript programs on a computer, but provides a different set of APIs, and the DOM API is not a core part of the Node.js runtime.

DOM & Javascript

- The DOM was designed to be independent
 - of any particular programming language,
 - making the structural representation of the document available from a single, consistent API.
- Even if most web developers will only use the DOM through JavaScript,
 - implementations of the DOM can be built for any language, e.g. Python example:

```
# Python DOM example
import xml.dom.minidom as m
doc = m.parse(r"C:\Projects\Py\chap1.xml")
doc.nodeName # DOM property of document object
p_list = doc.getElementsByTagName("para")
```

Accessing the DOM

- We don't have to do anything special to begin using the DOM.
 - We use the API directly in JavaScript.
- When you create a script,
 - whether inline in a `<script>` element or included in the web page,
 - you can immediately begin using the API for the document or window objects to manipulate the document itself, or any of the various elements in the web page. E.g.

```
<body onload="console.log('Welcome to my home page!');">
...
</body>
```
 - As it is generally not recommended to
 - mix the structure of the page (written in HTML) and manipulation of the DOM (written in JavaScript),
 - the JavaScript parts will be grouped together here, and separated from the HTML. E.g.
 - [10-create-html-element-at-runtime.html](#)

Accessing the DOM [10-create-html-element-at-runtime.html]

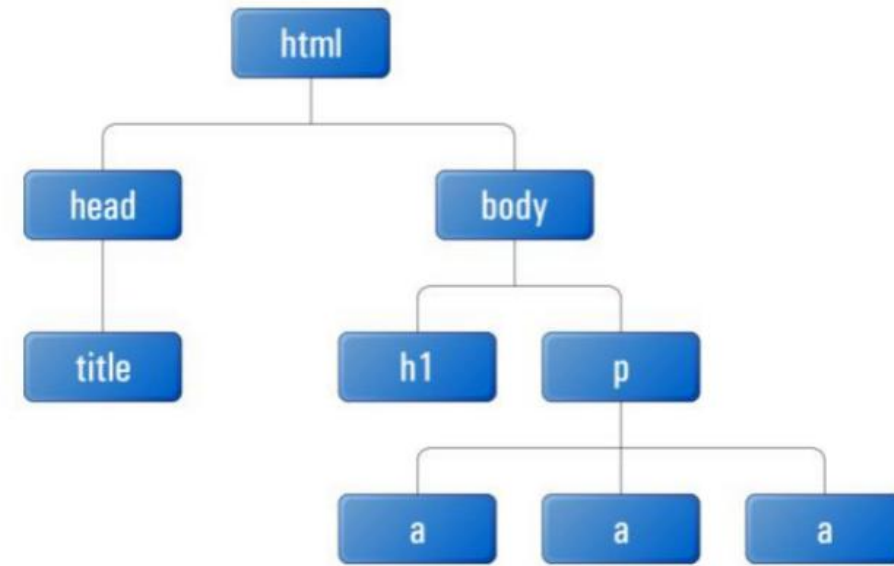
```
<html lang="en">
  <head>
    <script>
      // run this function when the document is loaded
      window.onload = () => {
        // create a couple of elements in an otherwise empty HTML page
        const heading = document.createElement("h1");
        const headingText = document.createTextNode("Big Head!");
        heading.appendChild(headingText);
        document.body.appendChild(heading);
      };
    </script>
  </head>
  <body></body>
</html>
```

Simple example

0-simple-page.html

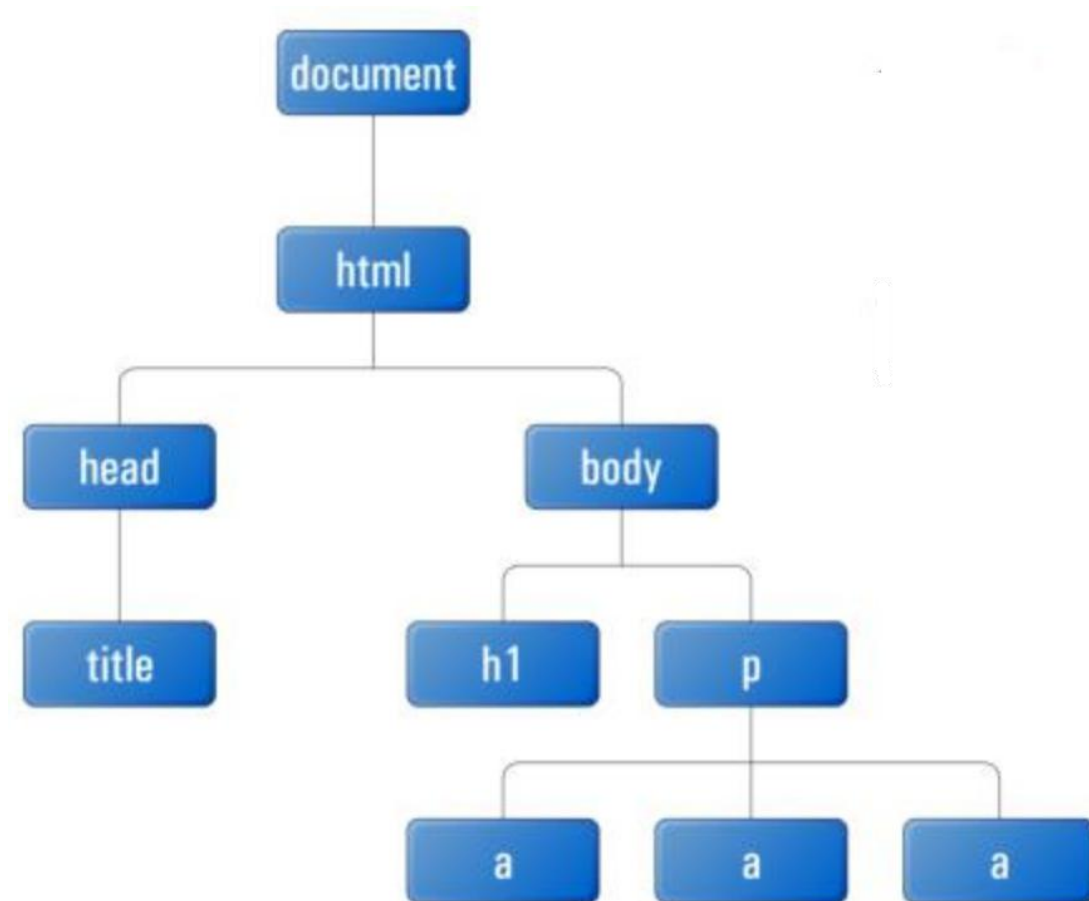
Nodes

- There are
 - many types of nodes in
 - the DOM document tree, representing
 - elements, text, comments, the document type declaration, etc.
- In DOM (graph), each HTML object is
 - represented by a node.
- These are element objects.
 - Every element is identified by its tag name.
 - H1, H2, P, HEAD, BODY, ...
- Moreover, there is a parent child relationship -
 - between the nodes.



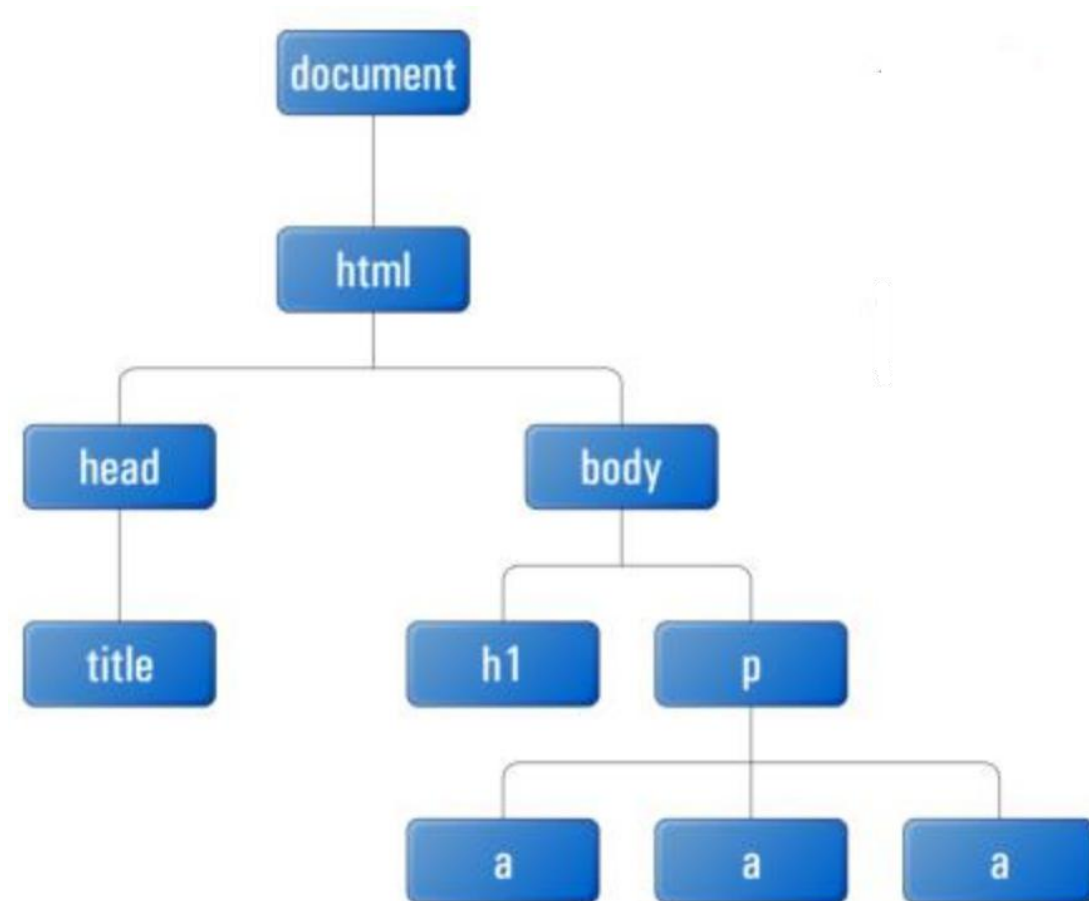
The DOCUMENT Node

- The DOCUMENT node, is
 - a special node;
 - always resides at the top of tree.



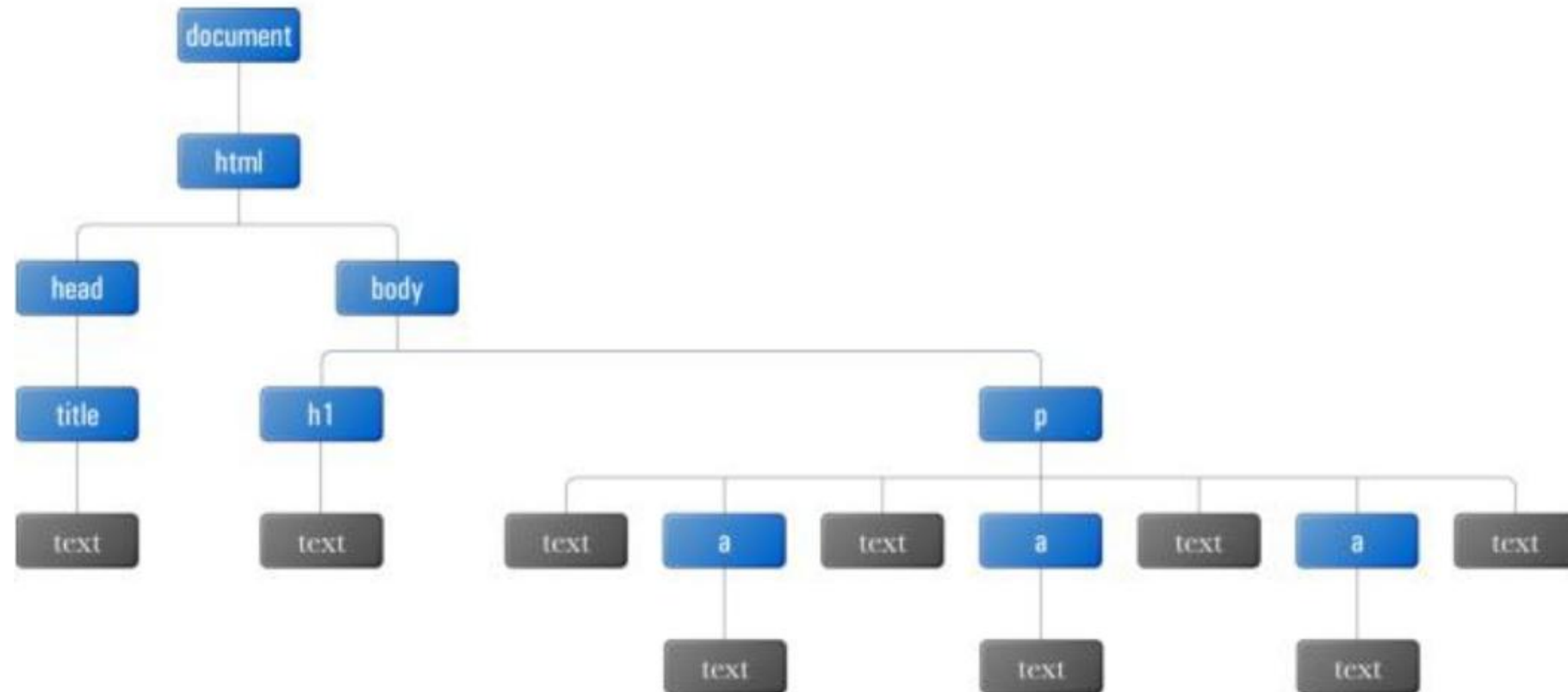
Types of Node

- Element node:
 - It points to the element itself, (not its content).
- Text node:
 - It is anything mentioned between
 - the angle brackets.
- Attribute node:
 - It is used to access
 - the attributes of a tag (e.g. href)



Text Node

- Each text node
 - has its own value, and
 - It is attached to an element node.

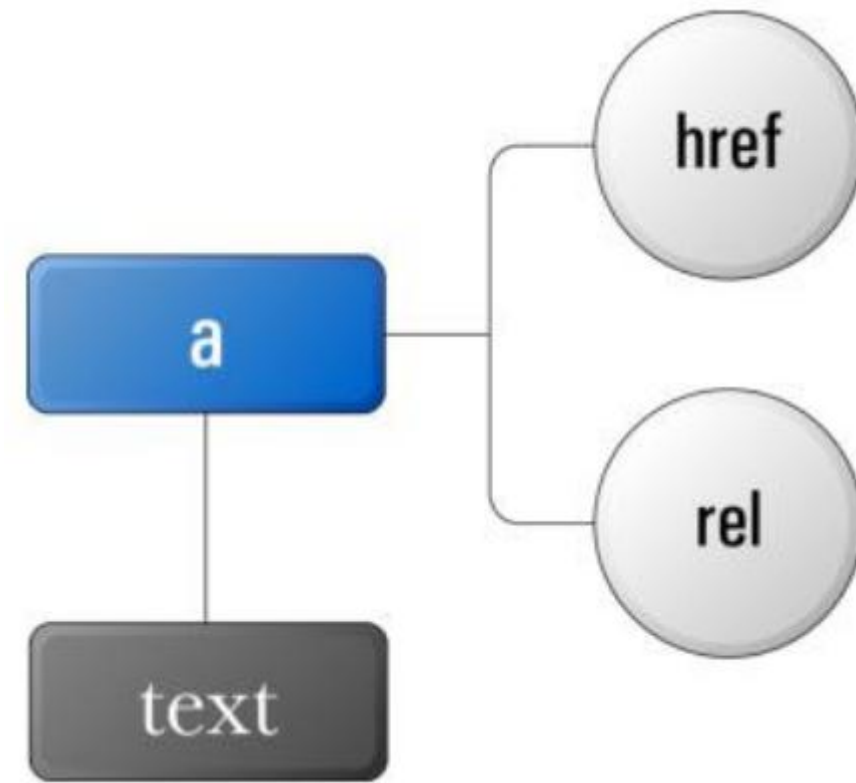


White space and text nodes

- White space may produce text nodes
 - Different browsers handle whitespace differently
- Never rely upon
 - the number and order of nodes when accessing the DOM

Attribute Node

- Attribute node
 - Refers to the attributes of the element
- Here -
 - The node 'a' refers an anchor element node,
 - with a 'text' node, and
 - two attribute nodes:
 - href
 - rel

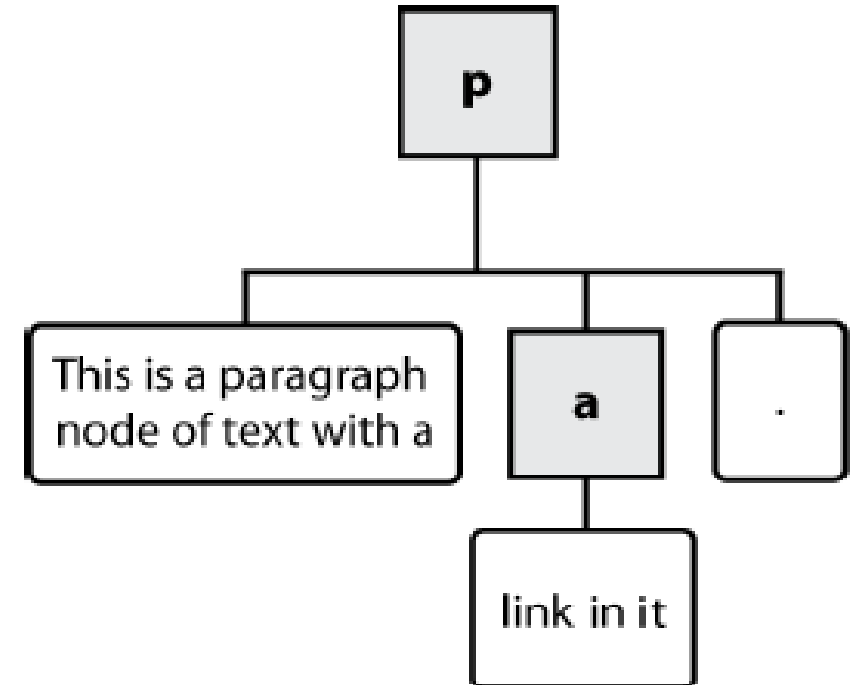
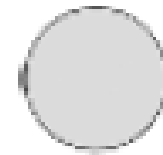
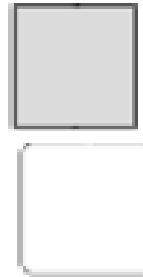


Attribute Node

```
<p>  
This is a paragraph of text with a  
<a href="/path/page.html">link in it</a>.  
</p>
```

HTML

- element nodes (HTML tag)
 - can have children and/or attributes
- text nodes (text in a block element)
- attribute nodes (attribute/value pair)
 - text/attributes are children in an element node
 - cannot have children or attributes
 - not usually shown when drawing the DOM tree



Attribute Node

- Find an element by a specific ID:

```
<p id = "element_id">
```

```
    Hello everyone.
```

```
</p>
```

- Use getElementById() method

```
var paragraph1 = document.getElementById("element_id")
```

Example: 1-GUI-page.html

Node id

- Once you have the reference to an element,
 - you have access to it's properties
- Example

```
var target = document.getElementById("paragraph1")  
alert(target.nodeName)
```

```
<p id = "paragraph1">Hello everyone.</p>
```

Core Interfaces of DOM

- The document and window objects
 - the objects whose interfaces you generally use most often in DOM programming.
- In simple terms,
 - the window object represents something like the browser,
 - the document object is the root of the document itself.
- Element inherits from the generic Node interface, and
 - together these two interfaces provide many of the methods and properties you use on individual elements.
 - These elements may also have specific interfaces for
 - dealing with the kind of data those elements hold, as
 - in the table object example in the previous section.

Core Interfaces of DOM

- A brief list of common APIs in web and XML page scripting using the DOM:
 - `document.querySelector()`
 - `document.querySelectorAll()`
 - `document.createElement()`
 - `Element.innerHTML`
 - `Element.setAttribute()`
 - `Element.getAttribute()`
 - `EventTarget.addEventListener()`
 - `HTMLElement.style`
 - `Node.appendChild()`
 - `window.onload`
 - `window.scrollTo()`

Traversing DOM tree

name(s)	description
firstChild, lastChild	start/end of this node's list of children
childNodes	array of all this node's children
nextSibling, previousSibling	neighboring nodes with the same parent
parentNode	the element that contains this node

Exercise

- Create a small HTML file with a paragraph in the body
 - Get a reference to the paragraph object
 - Use an alert() to display the node name
 - Use an alert() to display the object by itself

The document object

- It is used to refer nodes in the DOM of the page that the browser constructed
- `document`
 - The root of the tree is an object of type `HTMLDocument`
 - Using the global variable `document`,
 - we can access
 - all the nodes in the tree,
 - useful functions and
 - other global information such as -
 - title, referrer, domain, URL, body, images, links, forms, ...
 - open, write, close, getElementById, ...

The document object

- Example: 2-document-object.html
- `getElementById("radioLC")`
 - A method that makes use of the `id`
 - defined for any element in the page
 - An `id` must be unique in the page,
 - so only one element is ever returned by this function
 - The argument to `getElementById` specifies
 - which element is being requested

Using the DOM

- Example: 12-using-DOM.html

The document object

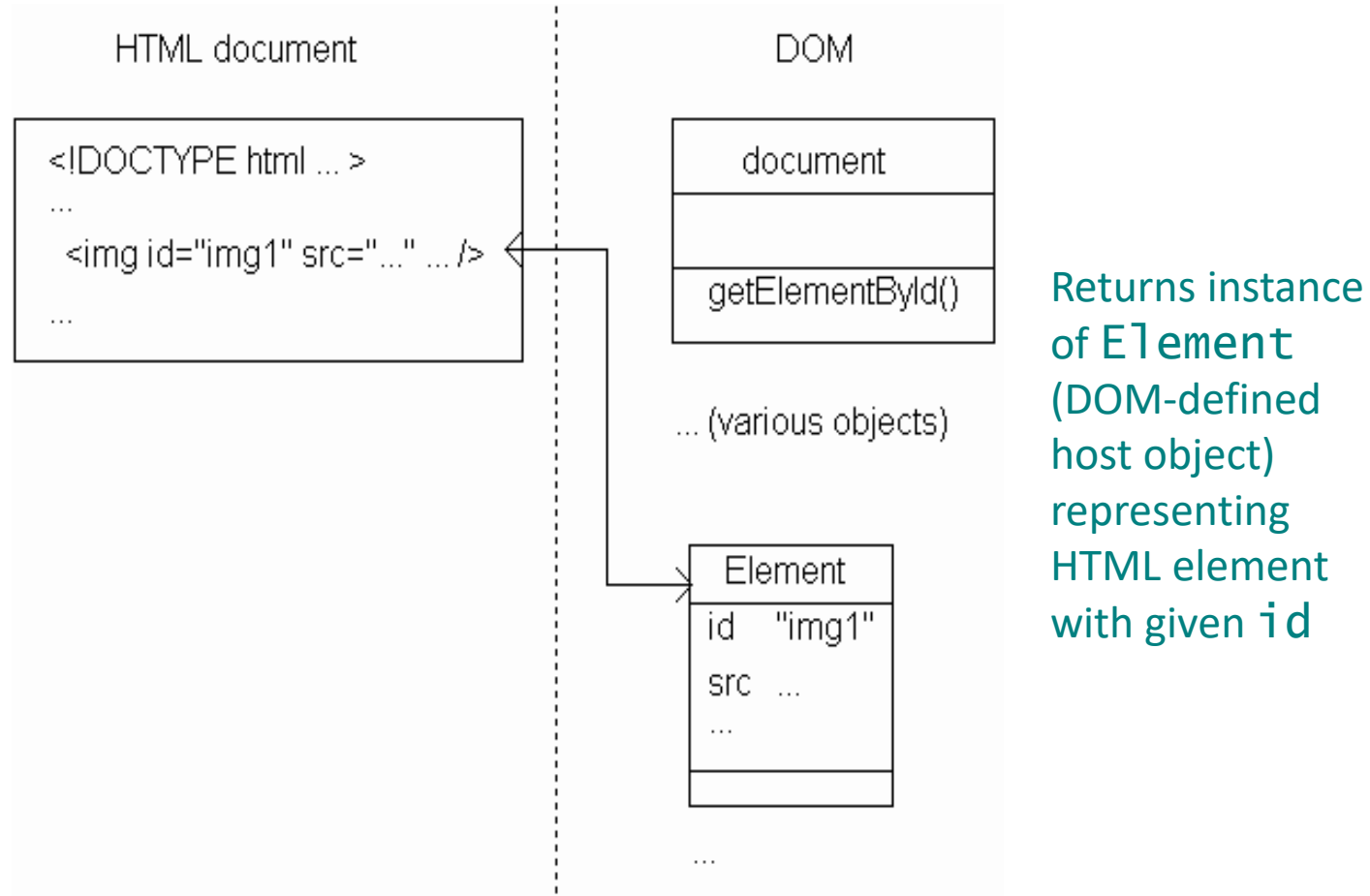
- Example: 3-information-about-elements.html

The document object

- Example: 4-dynamic-content.html

The document object

- Example: 5-on-mouse-over.html



The document object

- An event is
 - an occurrence of something potentially interesting to a script:
 - Ex: `mouseover` and `mouseout` events
- An HTML intrinsic event attribute is
 - used to specify a script to be called when an event occurs
 - Ex: `onmouseover`
 - Name of attribute is `on` followed by event name

Dynamically create elements

- Examples:
 - 10-create-html-element-at-runtime.html
 - 10-1-create-element-at-runtime.html

Intrinsic event handling

TABLE 5.1: HTML intrinsic event attributes.

Attribute	When Called
<code>onload</code>	Immediately after the body of document has been fully read and parsed by the browser (this attribute only pertains to <code>body</code> and <code>frameset</code>).
<code>onunload</code>	The browser is ready to load a new document in place of the current document (this attribute only pertains to <code>body</code> and <code>frameset</code>).
<code>onclick</code>	A mouse button has been clicked and released over the element.
<code>ondblclick</code>	The mouse has been double-clicked over the element.
<code>onmousedown</code>	The mouse has been clicked over the element.
<code>onmouseup</code>	The mouse has been released over the element.
<code>onmouseover</code>	The mouse has just moved over the element.
<code>onmousemove</code>	The mouse has moved from one location to another over the element.
<code>onmouseout</code>	The mouse has just moved away from the element.

Intrinsic event handling

<code>onfocus</code>	The element has just received the keyboard focus (this attribute only pertains to certain elements, including <code>a</code> , <code>label</code> , <code>input</code> , <code>select</code> , <code>textarea</code> , and <code>button</code>).
<code>onblur</code>	The element has just lost the keyboard focus (attribute pertains only to same elements as <code>onfocus</code>).
<code>onkeypress</code>	This element has the focus, and a key has been pressed and released.
<code>onkeydown</code>	This element has the focus, and a key has been pressed.
<code>onkeyup</code>	This element has the focus, and a key has been released.
<code>onsubmit</code>	This form element is ready to be submitted (only applies to <code>form</code> elements).
<code>onreset</code>	This form element is ready to be reset (only applies to <code>form</code> elements).
<code>onselect</code>	Text in this element has been selected (highlighted) in preparation for editing (applies only to <code>input</code> and <code>textarea</code> elements).
<code>onchange</code>	The value of this element has changed (applies only to <code>input</code> , <code>textarea</code> , and <code>select</code> elements).

Intrinsic event handling

- Example: 7-event-handling.html

Intrinsic event handling

- Example: 8-event-handling.html

Document tree: Node

TABLE 5.2: Non-method properties of `Node` instances.

Property	Description
<code>nodeType</code>	Number representing the type of node (<code>Element</code> , <code>Comment</code> , etc.). See Table 5.3.
<code>nodeName</code>	String providing a name for this <code>Node</code> (form of name depends on the <code>nodeType</code> ; see text).
<code>parentNode</code>	Reference to object that is this node's parent.
<code>childNodes</code>	Acts like a read-only array containing this node's child nodes. Has <code>length</code> 0 if this node has no children.
<code>previousSibling</code>	Previous sibling of this node, or <code>null</code> if no previous sibling exists.
<code>nextSibling</code>	Next sibling of this node, or <code>null</code> if no next sibling exists.
<code>attributes</code>	Acts like a read-only array containing <code>Attr</code> instances representing this node's attributes.

Document tree: Node types

Value	Symbolic Constant	Host Object Type
1	Node.ELEMENT_NODE	Element
2	Node.ATTRIBUTE_NODE	Attr
3	Node.TEXT_NODE	Text
8	Node.COMMENT_NODE	Comment
9	Node.DOCUMENT_NODE	Document
10	Node.DOCUMENT_TYPE_NODE	DocumentType

Document tree: Node types

TABLE 5.4: Method properties of Node instances.

Method	Functionality
<code>hasAttributes()</code>	Returns Boolean indicating whether or not this node has attributes.
<code>hasChildNodes()</code>	Returns Boolean indicating whether or not this node has children.
<code>appendChild(Node)</code>	Adds the argument Node to the end of the list of children of this node.
<code>insertBefore(Node, Node)</code>	Adds the first argument Node in the list of children of this node immediately before the second argument Node (or at end of child list if second argument is <code>null</code>).
<code>removeChild(Node)</code>	Removes the argument Node from this node's list of children.
<code>replaceChild(Node, Node)</code>	In the list of children of this node, replace the second argument Node with the first.

The document object

TABLE 5.5: Some properties of the `document` object.

Property	Value
<code>doctype</code>	An Object representing the document type declaration, if present, or <code>null</code> if not. Key properties are <code>publicId</code> and <code>systemId</code> , which are String values corresponding to the declaration's public and system identifier, respectively.
<code>title</code>	String representing the content of the <code>title</code> element (can be modified).
<code>body</code>	Object representing the body element of the document.
<code>cookie</code>	String representing the “cookies” associated with the current document; see Chap. 6 for more on cookies.
<code>URL</code>	String representing absolute URI for the document (read-only).
<code>domain</code>	String representing domain portion of URL, or <code>null</code> if a domain name is not available (read-only).
<code>referrer</code>	If this document was loaded because a hyperlink was clicked, this String is the URI of the page containing the hyperlink. Otherwise, it is the empty string.

The document object

<code>createElement(String)</code>	Given argument representing an element type name (such as <code>div</code>), returns an <code>Element</code> instance corresponding to the specified element type.
<code>createTextNode(String)</code>	Returns a <code>Text</code> instance containing the given <code>String</code> as its data value.
<code>getElementById(String)</code>	Given argument corresponding to the value of the <code>id</code> attribute of an element, returns that <code>Element</code> instance, or returns <code>null</code> if no document element has the specified <code>id</code> attribute value.
<code>getElementsByName(String)</code>	Given a <code>String</code> value representing an element type name, returns a “collection” (essentially an array) of <code>Element</code> instances corresponding to each element in the document having the given element type name.

The document object

TABLE 5.6: Some methods of **Element** instances.

Method	Purpose
<code>getAttribute(String)</code>	Returns value of attribute having name given by the <code>String</code> argument, or the empty string if no value (even a default) is available for the given attribute name.
<code>setAttribute(String, String)</code>	Creates an attribute with a name specified by the first argument <code>String</code> and assigns to it the value of the second argument <code>String</code> . If an attribute with this name already exists, it is overwritten with the new value specified, or an exception is thrown if the attribute is read-only (many host objects have read-only attributes).
<code>removeAttribute(String)</code>	Removes the specified attribute, or throws an exception if the attribute cannot be deleted (many host objects have attributes that cannot be deleted).
<code>hasAttribute(String)</code>	Returns Boolean value indicating whether or not the Element has an attribute with the specified name.
<code>getElementsByTagName(String)</code>	Like the method with the same name on document , but only returns those Element instances that are descendants of this Element .

Document tree: HTML properties

- Attribute values can be set in two ways:

```
element.setAttribute("id", "element3");  
element.id = "element3";
```

- As with CSS properties, former has some advantages:
 - Makes clear that setting an HTML attribute, not merely a property of an object
 - Avoids certain special cases, e.g.
 - `element.setAttribute("class", "warning");`
 - `element.className = "warning";`

`document.getElementsByTagName()`

- Example: 9-getElementsByTagName.html

`document.getElementsByTagName()`

- Example: 9-getElementsByTagName.html

Using event listeners

- Example:
 - 13-button-click-1.html

Using event listeners

- Example:
 - 14-button-click-2.html

Removing event listener

- If you've added an event handler using `addEventListener()`,
 - you can remove it again using the `removeEventListener()` method.
- Example:

```
btn.removeEventListener("click", changeBackground);
```

Adding multiple event listeners

- By making more than one call to `addEventListener()`,
 - providing different handlers,
 - you can have multiple handlers for a single event:

```
myElement.addEventListener("click", functionA);  
myElement.addEventListener("click", functionB);
```

- Both functions would now run when the element is clicked.